

Passly

Informe de la Obligatoria 1

Desarrollo de Aplicaciones II - Comision Lunes TM

Fecha de entrega: 14/09/2026

Equipo: Federico Torcini, Simon Forteza, Juan Segundo Addamo y Lucio Dillon

Version de trabajo: corte verificable del 14/09/2026 ([main 8b096ef](#))

Este documento diferencia lo implementado de lo planificado y declara de forma explicita las deudas tecnicas que permanecen abiertas.

Resumen ejecutivo

Passly es una plataforma para vender y validar entradas de eventos. El backend adopta un monolito modular: una sola aplicacion desplegable, dividida en componentes de negocio con contratos explicitos y fronteras verificadas mediante Spring Modulith.

Al corte de este informe estan integrados Usuarios, Productoras, Eventos, Seguridad, Ventas y Pagos. Ventas implementa el componente stateful y el patron Facade; Pagos implementa el patron Adapter REST. La app web ya cubre identidad, alta y padron de productoras, cartelera y backoffice de eventos. Permanecen pendientes la emision de tickets, las pantallas de carrito y ordenes, y el cableado de Ventas con el Adapter real de Pagos.

1. Problema, alcance y arquitectura

Passly cubre el recorrido que comienza cuando un organizador publica un evento y termina cuando un validador acepta un ticket en la puerta. Un comprador selecciona entradas, realiza un pago, recibe un ticket con QR y posteriormente ese QR se marca como utilizado para impedir el doble ingreso.

La Obligatoria 1 se concentra en demostrar decisiones de arquitectura defendibles y funcionalidad incremental. La cantidad de código no reemplaza la necesidad de justificar componentes, estados, patrones, seguridad y fronteras.

1.1 Monolito modular

El backend se ejecuta como un único artefacto Spring Boot, pero cada componente publica contratos propios y mantiene controladores, implementaciones, entidades y repositorios bajo `internal`. Los intercambios entre componentes utilizan interfaces y DTO; no se importan internos ajenos.

Spring Modulith analiza las dependencias declaradas y hace fallar el build ante accesos indebidos o ciclos. Esta decisión conserva transacciones locales y evita introducir sagas, fallos de red y despliegues coordinados antes de que exista una necesidad real. La decisión completa se registra en ADR-001.

1.2 Persistencia por componente

Se utiliza una instancia PostgreSQL con esquemas separados para eventos, usuarios, productoras, ventas y pagos. Cada componente es propietario de sus tablas. Las referencias a datos ajenos son identificadores lógicos, sin joins ni claves foráneas entre esquemas, y se validan mediante el contrato público del propietario.

Este enfoque reduce complejidad operativa y mantiene visible la propiedad de los datos. El detalle se registra en ADR-002.

1.3 Tecnologías principales

- Java 21, Spring Boot 4.1.1 y Spring Modulith 2.1.1.
- Spring Data JPA, Hibernate y PostgreSQL.
- React, Vite y TypeScript para la app web.
- Maven y GitHub Actions para build y verificación de fronteras.
- Docker Compose para la infraestructura local de demostración.

2. Componentes y contratos

2.1 Usuarios, Productoras y Eventos

Usuarios administra registro, identidades, credenciales BCrypt y roles globales COMPRADOR, ORGANIZADOR, VALIDADOR y ADMIN. El hash no forma parte de `UsuarioDTO`. **Productoras** administra organizaciones, miembros y roles internos DUENIO, STAFF y VALIDADOR; sus endpoints obtienen el actor desde Spring Security, sin headers de identidad propios.

Eventos administra alta, publicacion, tipos de entrada, precios, cupos y disponibilidad. Cada evento pertenece a una productora. Para autorizar acciones delega en `ProductoraService`, y para las operaciones de compra expone disponibilidad y descuento de cupo mediante su contrato publico.

2.2 Seguridad

Seguridad configura HTTP Basic, autenticacion stateless y autorizacion declarativa mediante `@PreAuthorize`. Reutiliza el contrato de autenticacion de Usuarios y representa el id numerico dentro del principal. Asi cada controlador identifica al actor sin importar repositorios ni internos de Usuarios.

2.3 Ventas - stateful y Facade

Ventas administra el carrito y la confirmacion de ordenes. `CarritoDeCompra` tiene alcance de sesion (`@SessionScope`), conserva la seleccion durante la conversacion HTTP y expira luego de cinco minutos. `VentaService` actua como Facade: ofrece operaciones simples y oculta la coordinacion con Usuarios, Eventos, la pasarela de cobro y la persistencia propia.

En el corte actual Ventas cobra mediante `PasarelaDePagoSimulada`. El puerto ya existe, pero reemplazar esa implementacion por el `PagoService` real es una deuda de integracion declarada.

2.4 Pagos - Adapter REST

Pagos expone `PagoService` como contrato estable del dominio y adapta las solicitudes y respuestas de una pasarela REST externa. Persiste operaciones y comprobantes en su propio esquema. La URL externa es configurable y el Adapter evita propagar modelos del proveedor al resto del sistema.

2.5 Componentes posteriores

Tickets emitira entradas con QR; Accesos validara su uso unico; Notificaciones enviara mensajes; Facturacion adaptara una integracion SOAP. No estan implementados en este corte y no se contabilizan como requisitos cumplidos.

3. Capas, estado y patrones

3.1 Arquitectura en tres capas

Los componentes de negocio siguen el flujo presentacion -> negocio -> datos.

- **Presentacion:** traduce HTTP a solicitudes de negocio, valida la entrada y transforma errores en respuestas consistentes.
- **Negocio:** contiene reglas, orquestacion, autorizacion de alcance y limites transaccionales.
- **Datos:** implementa persistencia mediante entidades JPA y repositorios Spring Data.

Las entidades no cruzan la frontera del componente. Los intercambios se realizan mediante contratos y DTO.

3.2 Stateful y stateless

Usuarios, Productoras, Eventos, Seguridad y Pagos no conservan estado conversacional entre llamadas. Persistir datos de dominio no los vuelve stateful: ese estado vive en PostgreSQL. La autenticacion tambien es stateless, porque cada request vuelve a presentar sus credenciales.

Ventas es stateful por su carrito con alcance de sesion. La cookie de sesion identifica la conversacion del comprador, aunque la autenticacion siga siendo HTTP Basic. El carrito mantiene items temporalmente y aplica un vencimiento absoluto de cinco minutos desde su creacion; el contenedor destruye la instancia cuando finaliza la sesion.

3.3 Patrones implementados

DAO

Los repositorios Spring Data separan la logica de negocio del acceso a PostgreSQL. Los servicios no escriben SQL ni administran conexiones.

Facade

VentaService concentra las operaciones de carrito y compra, y oculta al cliente la coordinacion de disponibilidad, identidad, cobro y orden. La emision de tickets se incorporara detras de esta misma frontera.

Adapter

PagoService define el lenguaje interno de pagos y su implementacion REST traduce hacia el contrato de la pasarela. Esto permite cambiar el proveedor o simularlo sin contaminar Ventas con detalles externos.

4. Seguridad, transacciones e integraciones

4.1 Autenticacion y autorizacion

Usuarios valida el registro, rechaza emails repetidos y persiste hashes BCrypt. El login web valida email y password mediante GET /api/usuarios/me; no existe un endpoint especial de login ni JWT. Un DaoAuthenticationProvider autentica cada request.

La autorizacion combina dos ejes: rol global y alcance dentro de una productora. Un alta anonima solo puede registrar COMPRADOR; los roles privilegiados requieren ADMIN. Solo un ORGANIZADOR con membresia adecuada puede crear o publicar eventos. Los errores distinguen 401 para falta de autenticacion y 403 para permisos insuficientes.

4.2 Transaccion critica implementada

La confirmacion cobra primero mediante el puerto de la pasarela y luego abre la transaccion local en ConfirmacionDeCompra. Dentro de ella se descuenta el cupo y se persiste la orden. Si falla la parte local, se ejecuta una compensacion de cobro de mejor esfuerzo.

Esta frontera evita mantener una conexion de base abierta durante una llamada externa. No se afirma atomicidad distribuida: la compensacion puede fallar y requiere tratamiento posterior. La emision de tickets aun no forma parte de la transaccion porque ese componente no esta implementado.

4.3 Integraciones

- REST saliente implementado en Pagos hacia la pasarela configurable.
- Integracion pendiente entre Ventas y el PagoService real; hoy usa una implementacion simulada local.
- Cola punto a punto orden.pagada prevista para Facturacion.
- Topico ticket.emitido previsto para Notificaciones y Accesos.
- SOAP previsto para un servicio AFIP simulado.

5. Verificacion y demostracion

5.1 Evidencia automatica

EstructuraDeModulosTest verifica ausencia de violaciones, Usuarios como raiz, el grafo aciclico con Ventas en la cima y la generacion de documentacion desde el codigo. El workflow `modulith-boundaries.yml` ejecuta esta verificacion en GitHub Actions; el commit de corte `8b096ef` finalizo correctamente.

La suite completa requiere PostgreSQL. En esta revision local no se certifica una corrida integral contra la base porque Docker no estaba disponible; esta limitacion se declara para no confundir compilacion o CI de fronteras con una prueba end-to-end.

5.2 App web integrada

- **PAS-14:** esqueleto React, router, cliente HTTP, manejo de errores y guardas de rutas.
- **PAS-15:** registro, login, perfil, logout y alta administrativa de roles privilegiados.
- **PAS-16:** alta y listado de productoras, backoffice y padron de miembros.
- **PAS-17:** cartelera y detalle publicos, alta, listado y publicacion de eventos por productora.
- **Pendiente:** pantallas de carrito y mis ordenes.

5.3 Guion de demostracion

- 1 Levantar PostgreSQL con `docker compose up -d` e iniciar el backend con el perfil demo.
- 2 Consultar healthcheck y modelo de Modulith; luego iniciar el frontend.
- 3 Demostrar registro/login, perfil, alta de productora y gestion de su padron.
- 4 Crear y publicar un evento; verificar cartelera completa, filtro y detalle.
- 5 Ejecutar los casos HTTP de 401, 403 y aislamiento entre productoras.
- 6 Probar carrito y confirmacion por API, verificando su sesion y vencimiento.
- 7 Ejecutar la suite completa con PostgreSQL disponible y conservar el resultado.

5.4 Pendientes para cerrar el producto

- Cablear Ventas con el Adapter real de Pagos y probar la compensacion.
- Implementar Tickets y su emision posterior a la compra.
- Completar carrito y mis ordenes en la app web.
- Ejecutar y registrar la demo integral en la maquina de la defensa.
- Realizar una revision cruzada para que cada integrante pueda explicar cualquier componente.

6. Uso de inteligencia artificial y conclusiones

6.1 Declaracion obligatoria de uso de IA

El equipo declara que utilizo herramientas de inteligencia artificial durante el desarrollo de Passly. Su uso se informa con fines de transparencia academica y no sustituye el trabajo, la revision ni la responsabilidad de los integrantes.

Herramientas utilizadas

- **Claude (Anthropic):** asistencia para analizar la consigna, estructurar el roadmap, dividir el trabajo en issues y apoyar la implementacion y documentacion tecnica.
- **Codex (OpenAI):** inspeccion del repositorio e historial, contraste del estado integrado, apoyo a la implementacion, redaccion de ADR e informe, generacion reproducible del PDF y controles de consistencia y presentacion.

Alcance del uso

Las herramientas intervinieron en la interpretacion de requisitos, evaluacion de alternativas, planificacion, apoyo a la escritura y revision de codigo, preparacion de pruebas y documentacion. Las decisiones finales de arquitectura, el codigo aceptado y el contenido presentado fueron seleccionados por el equipo.

Procesos de validacion

Los resultados sugeridos fueron revisados por integrantes, contrastados con la consigna, el codigo integrado y documentacion oficial, y corregidos cuando no coincidian. El codigo se valida mediante compilacion, pruebas de fronteras, pruebas de API y demostraciones contra PostgreSQL. La documentacion se compara con `main`, se revisa mediante pull requests y el PDF se inspecciona visualmente. Ningun resultado se considera valido solo por haber sido producido por una IA.

Responsabilidad sobre el contenido

Los integrantes asumen plena responsabilidad academica por el codigo, las decisiones y la documentacion presentados. Cada miembro debe comprenderlos, explicar sus fundamentos y defender cualquier parte durante la evaluacion oral, aunque haya recibido asistencia de IA.

La declaracion tiene fines de transparencia y no implica por si misma una penalizacion. La omision del uso de estas herramientas o la presentacion de informacion falsa o enganosa puede considerarse una falta a la integridad academica. Antes de entregar, cada integrante debe confirmar que la lista incluye todas las herramientas realmente utilizadas y que el alcance informado es correcto.

6.2 Conclusiones

El monolito modular equilibra separacion y simplicidad: conserva contratos claros, transacciones locales y una demostracion reproducible sin asumir prematuramente los costos de un sistema distribuido. El esquema PostgreSQL por componente refleja la misma propiedad de datos.

El corte ya demuestra seis modulos backend, autenticacion y autorizacion, un componente stateful y los patrones DAO, Facade y Adapter. El frontend ofrece recorridos reales de identidad, productoras y eventos. Las deudas se mantienen visibles: integrar Ventas con Pagos, emitir tickets, completar las pantallas de compra y ejecutar la verificacion integral con PostgreSQL antes de la defensa.

Referencias

- Repositorio Passly: <https://github.com/SimonForteza/Passly>
- Spring Boot: <https://docs.spring.io/spring-boot/index.html>
- Spring Modulith: <https://docs.spring.io/spring-modulith/reference/index.html>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Documentacion interna: `CLAUDE.md`